Internet Applications Design and Implementation

Week 10 - Client-side rendering. React State Management.

React

useState

Revisions from week 9

```
import { useState } from "react";

export default function Counter() {
    const [count, setCount] = useState(0);

    return (
        <div>
            <h2>{count}</h2>
            <button onClick={() => setCount(count - 1)}>-</button>
            <button onClick={() => setCount(count + 1)}>+</button>
        </div>
    );
}
```

what if the initial value is not known in compile time?

React
useState

```
function calculateInitialValue() {
    console.log("calculateInitialValue called");
    return 0;
}

export default function Counter() {
    const [count, setCount] = useState(calculateInitialValue);

    return (
        <div>
            <h2>{count}</h2>
            <button onClick={() => setCount(count - 1)}>-</button>
            <button onClick={() => setCount(count + 1)}>+</button>
        </div>
    );
}
```

lazy initialization: React calls the function once in the first rendering. The console.log fires only once.

```
const [count, setCount] = useState(calculateInitialValue());
```

the function runs on every render, even though useState ignores the result after the first render. The console.log fires on every re-render

React

Props/State

Revisions from week 9

Each react component may have:

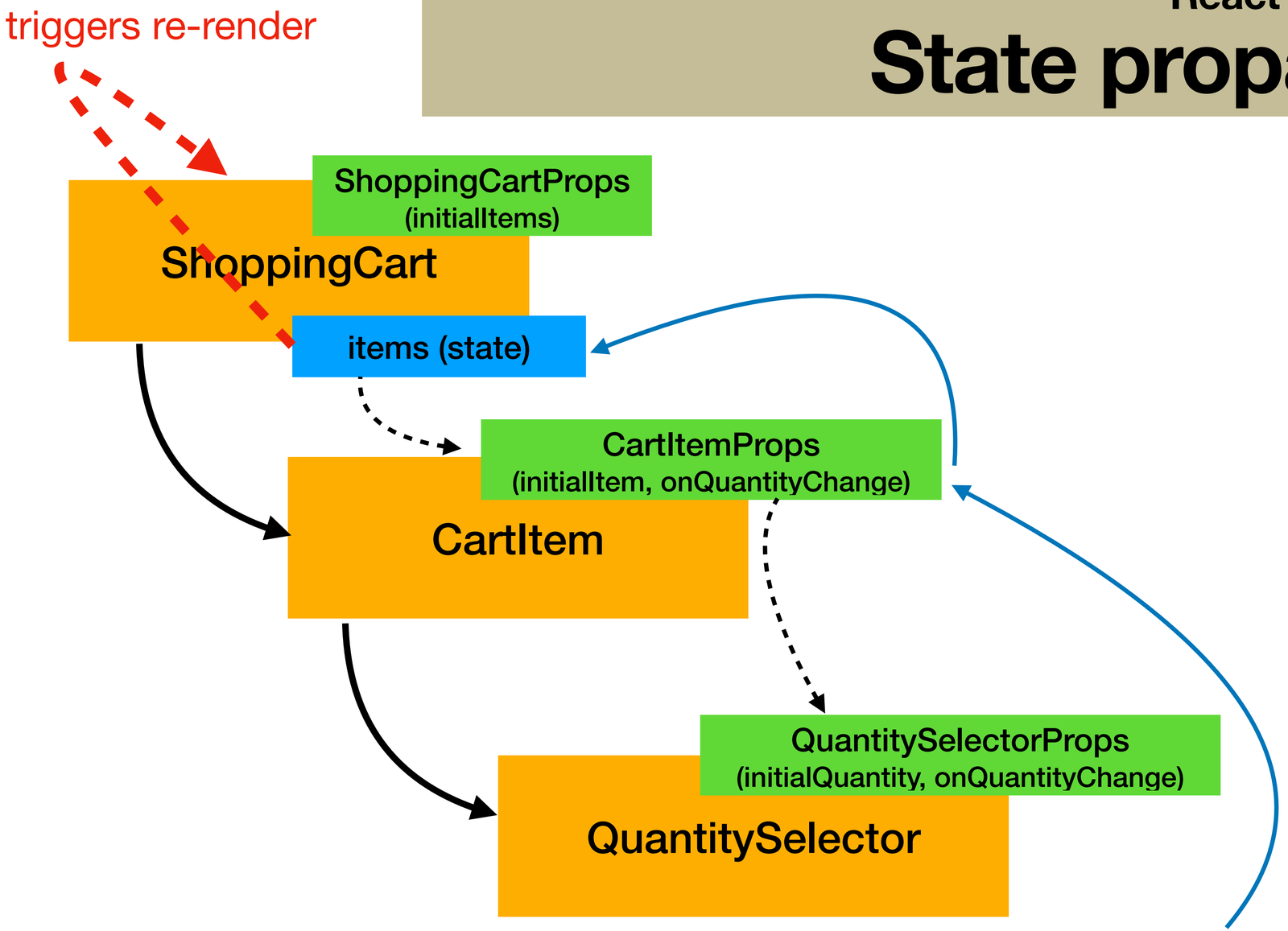
- Immutable properties - “**props**”
- Mutable properties - “**state**”

```
interface CartItemProps {
    readonly initialQuantity: number
}
```

```
export default function CartItem( props: CartItemProps ) {

    const [quantity, setQuantity] = useState<number>( props.initialQuantity );

    return (
        <div>
            ...
            <QuantitySelector
                quantity={ quantity }
                onChange={(newQuantity) => setQuantity(newQuantity) } />
        </div>
    );
}
```

React

State propagation




```
<button onClick={() => onQuantityChange(quantity-1)}>-</button>
```

React

State propagation

```
export default function ShoppingCart(props: ShoppingCartProps) {

   const [items, setItems] = useState<CartItemModel[]>(props.initialItems);

   function handleQuantityChange(id: string, newQuantity: number) {
       setItems(...);
   }

   <CartItem {…item} onQuantityChange={(qty) => handleQuantityChange(item.id, qty)} />
   ...
   <Subtotal items={items} />

}


export default function CartItem(props: CartItemProps) {
    return (
       ...
       <QuantitySelector quantity={props.quantity}
                         onChange={props.onQuantityChange} />
       ...
   );
}


export default function QuantitySelector(
    { quantity, onQuantityChange }: QuantitySelectorProps): ReactNode {

    <button onClick={() => onQuantityChange(quantity-1)}>-</button>

}
```

React

Context API

👎 Prop drilling + callbacks doesn’t scale to large component trees

The Context API provides a means to share values like state, functions, or any data across the component tree without passing props down manually at every level.

1. Create a context using `createContext(type)`
2. Wrap the component tree with the `context.provider` and an object of type `type` (which is the object that you want to share)
3. Consume the context only on those components that need it, using the react hook `useContext()`

React

Context API

```tsx
type CounterContextType = {
    count: number;
    increment: () => void;
};

export const CounterContext = createContext<CounterContextType | null>(null);

export default function App() {
    const [count, setCount] = useState(0)

    function increment() { setCount(count + 1);}

    return (
        <CounterContext.Provider value={{ count, increment }}>
            <CounterDisplay />
            <CounterButton />
        </CounterContext.Provider>
    );
}
```

provide a value for the context that will be propagated to all consumers

Wrap the component tree with a Context Provider

Note: in this case, the value is tied to the state (if it changes, the whole tree is re-rendered)

React

Context API

```
import {useContext} from "react";
import {CounterContext} from "../App.tsx";

export default function CounterDisplay() {
    const { count } = useContext(CounterContext)!;
    return <h2>Count: {count}</h2>;
}
```

```
import {useContext} from "react";
import {CounterContext} from "../App.tsx";

export default CounterButton() {
    const { increment } = useContext(CounterContext)!;
    return <button onClick={increment}>Increment</button>;
}
```

“Look ma, no props!”

React

State Propagation

Exercise

	Prop Drilling	Context API
Data passing (function parameters/shared object)		
Data flow (implicit/explicit)		
Propagation can bypass intermediate components (yes/no)		
Scalability in deep trees (good/poor)		
Debugging (easier/harder)		
Component coupling (tight/loose)		

React

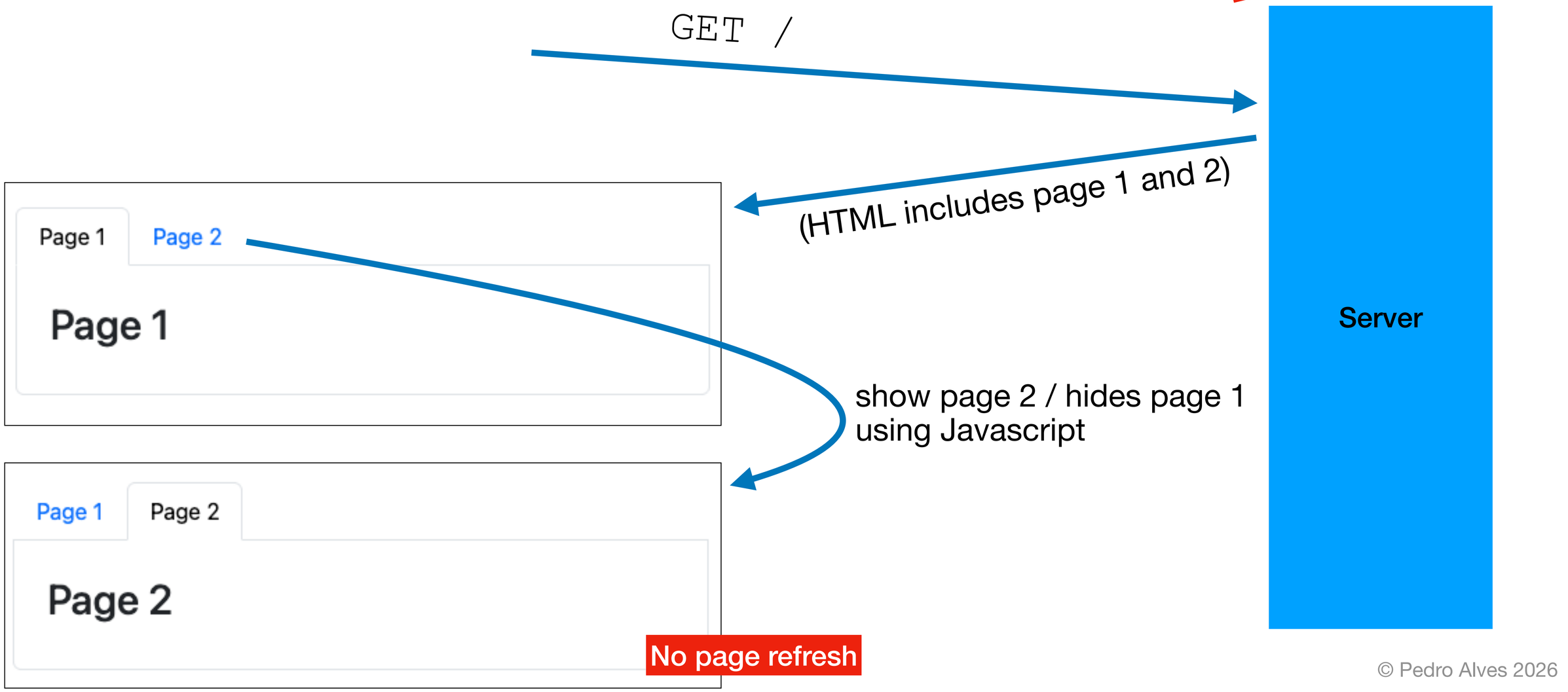
State Propagation

	Prop Drilling	Context API
Data passing (function parameters/shared object)	function parameters	shared object
Data flow (implicit/explicit)	explicit	implicit
Propagation can bypass intermediate components (yes/no)	no	yes
Scalability in deep trees (good/poor)	poor	good
Debugging (easier/harder)	easier	harder
Component coupling (tight/loose)	tight	loose

Navigation

Single Page Application (SPA)

Revisions from week 3



Client-Side Navigation

Revisions from week 1

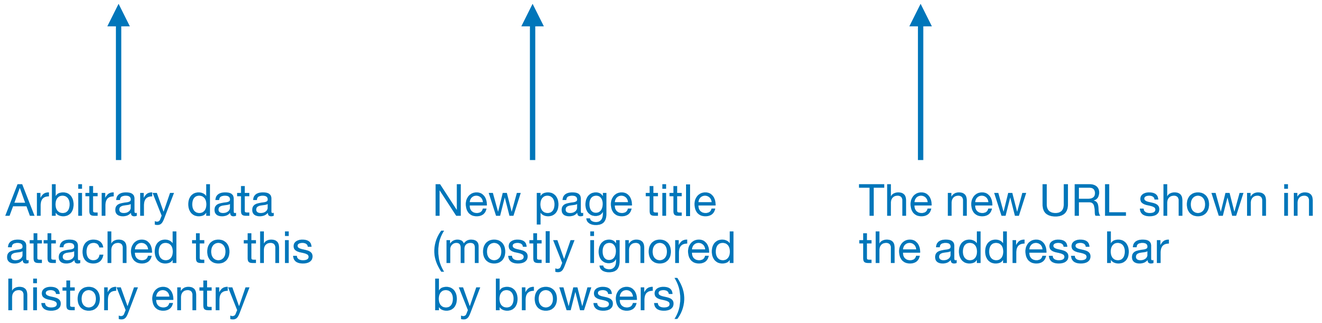

```
<h1 id="h"></h1>
<a href="/1" onclick="go(event,'/1')">Page 1</a>
<a href="/2" onclick="go(event,'/2')">Page 2</a>

<script>
    const h = document.getElementById('h');

    function render() {
        h.textContent = location.pathname === '/2' ? 'Page 2' : 'Page 1';
    }

    function go(e, path) {
        e.preventDefault();
        history.pushState(null, '', path);
        render();
    }

    addEventListener('popstate', render); // back/forward
    render(); // initial
</script>
```

Client-Side Navigation

History API

Allows updating the browser URL without reloading the page

```
history.pushState( state , title , url )
```



Each `pushState` call creates a new history entry (back and forward work, bookmarkable)

Client-Side Navigation

Handling back/forward navigation

Even though back/forward change the URL, the page must be re-rendered

```
addEventListener('popstate', render);
```

This event is triggered when user clicks Back or Forward or navigates browser history

This function should inspect the current URL and render the page appropriately

```
function render() {
    if (location.pathname === '/1') {
        h.textContent = 'Page 1';
    } else if (location.pathname === '/2') {
        h.textContent = 'Page 2';
    } else {
        h.textContent = 'Unknown Page';
    }
}
```

Client-side navigation

Even with the History API, it’s not easy to:

- Manage all the routes (especially nested)
- Rendering the correct page based on the current URL
- Preserving shared layouts across navigable pages
- Handling redirects
- …

React Router

React Router is a library built on top the History API with:

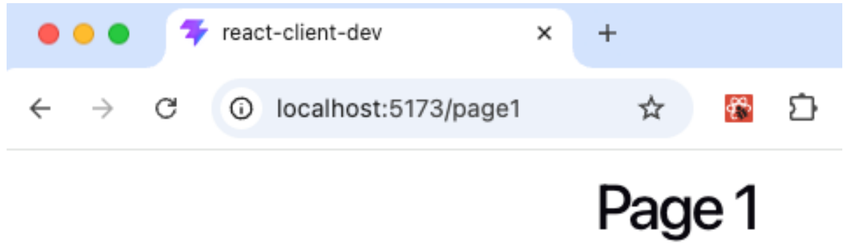
- Declarative navigation
- Route -> Component mapping
- Nested layouts
- Navigation state management

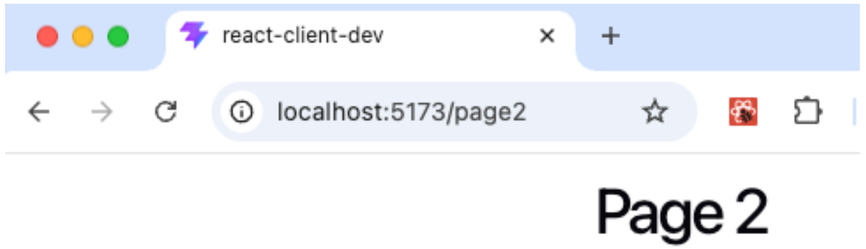
Install it with `pnpm add react-router-dom`

https://reactrouter.com/start/declarative/routing

React Router

```
import {BrowserRouter, Routes, Route} from "react-router";
...

export default function App() {
    return (
        <BrowserRouter>
            <Routes>
                <Route path="/page1" element={<Page1/>}/>
                <Route path="/page2" element={<Page2/>}/>
                <Route path="*" element={<Home/>}/>
            </Routes>
        </BrowserRouter>
    )
}
```





Routes map URLs into React Components

React Router

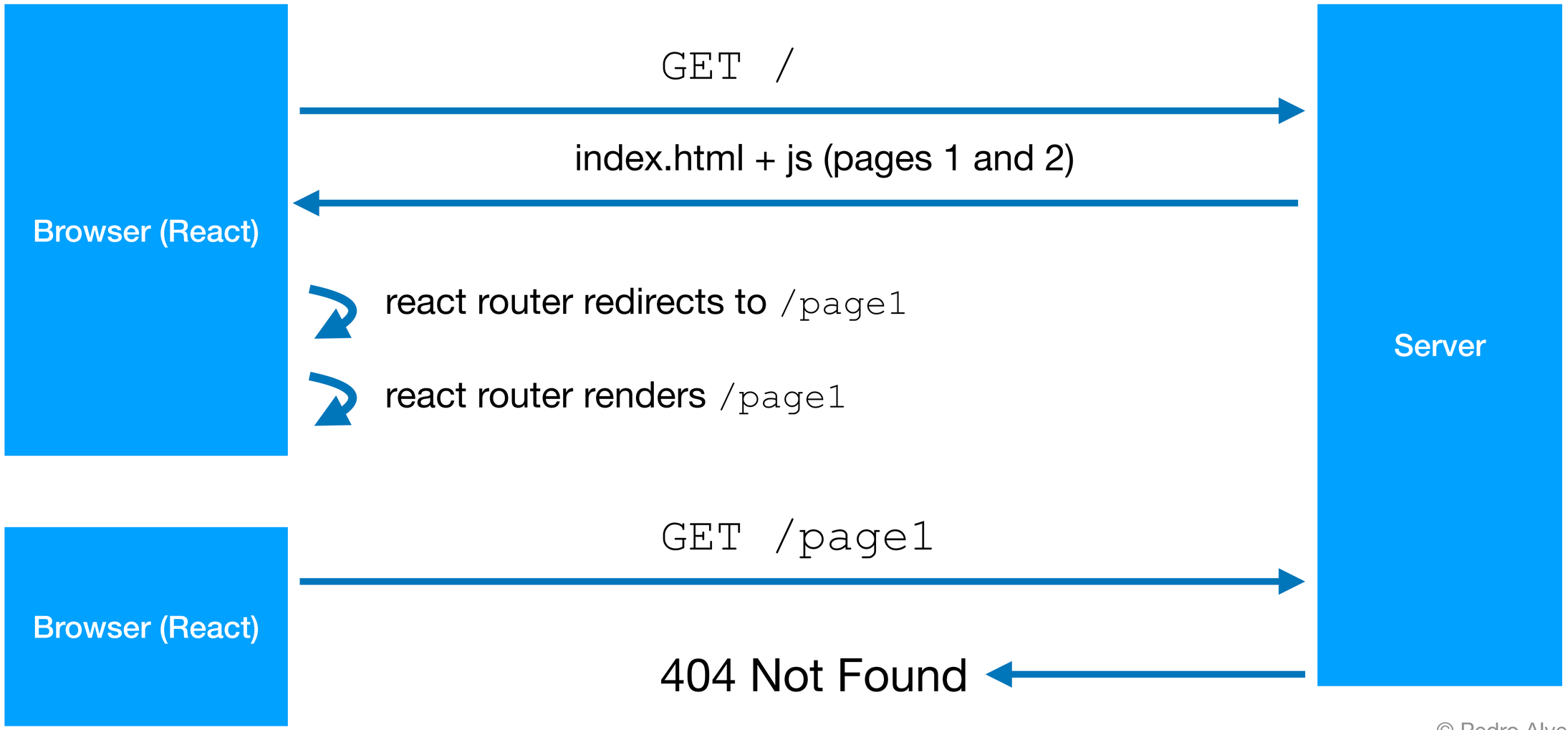
Conditional routing

- User navigates to /login
  - With sessionUser (user already authenticated) -> redirect to /dashboard
  - Without sessionUser -> show login component

- User navigates to /dashboard
  - Without sessionUser (user not authenticated) -> redirect to /login
  - With sessionUser -> show dashboard component

```
<Route path="/login"     element={sessionUser ? <Navigate to="/dashboard"/> : <Login/>}/>
<Route path="/dashboard" element={sessionUser ? <Dashboard/> : <Navigate to="/login"/>}/>
```

Double routing problem



Double routing problem

In CSR applications, there is a conflict between:

- backend routes - resolved by the server through HTTP requests
- frontend routes - resolved in the browser through the History API

The problem appears when the browser requests a frontend route to the server (which doesn’t exist)

Double routing problem

Server fallback to index.html

All frontend routes are resolved by the server to index.html

```
/page1 -> index.html
/page2 -> index.html
```

Separate backend and frontend routes

```
/app/page1 -> index.html
/app/page2 -> index.html
/api/users -> normal API route
/web/landing-page -> normal SSR route
```

Double routing problem

In a Spring application:

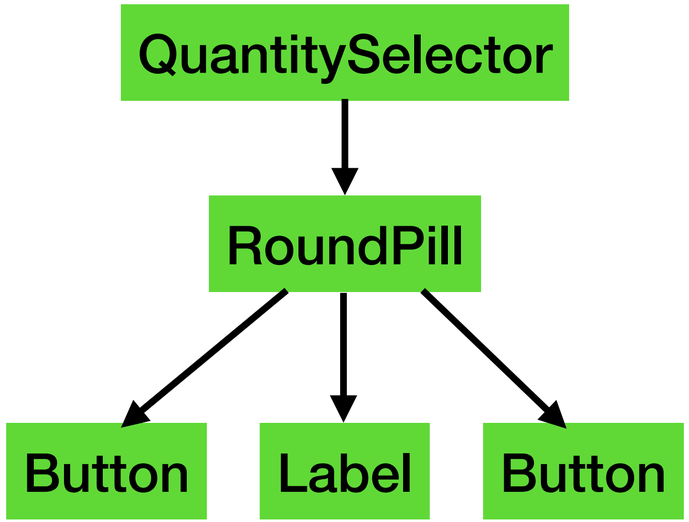
1. Copy index.html and CSR js files to /src/resources/static/app
2. Add this configuration to respond with index.html for all /app/* requests

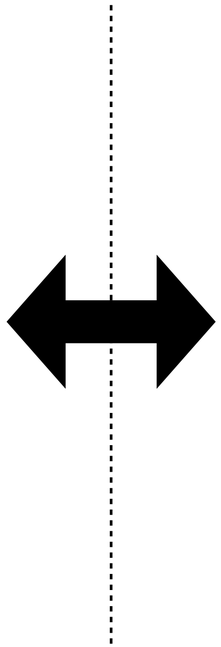
```
@Configuration
class MvcConfig() : WebMvcConfigurer {

    override fun addResourceHandlers(registry: ResourceHandlerRegistry) {
        registry.addResourceHandler("/app/**")
            .addResourceLocations("classpath:/static/app/")
            .resourceChain(true)
            .addResolver(object : PathResourceResolver() {
                override fun getResource(resourcePath: String, location: Resource): Resource? =
                    super.getResource(resourcePath, location)
                        ?: location.createRelative("index.html")
            })
    }
}
```

Component rendering

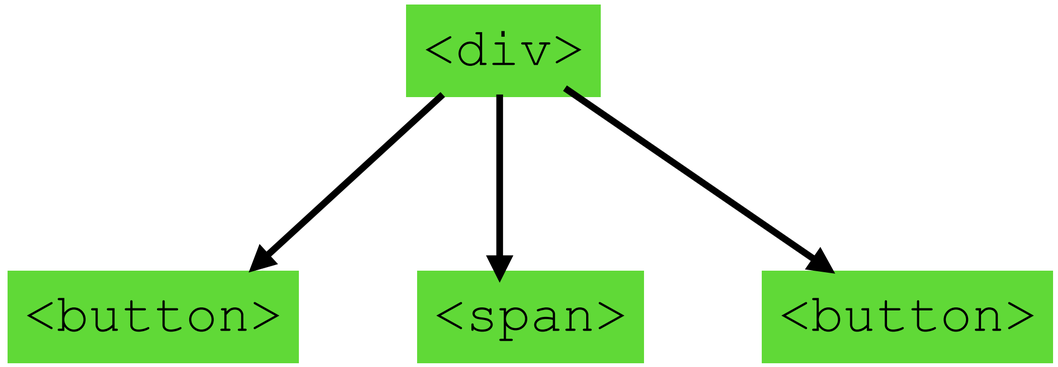
Revisions from week 9

Component Tree (*)





DOM Tree

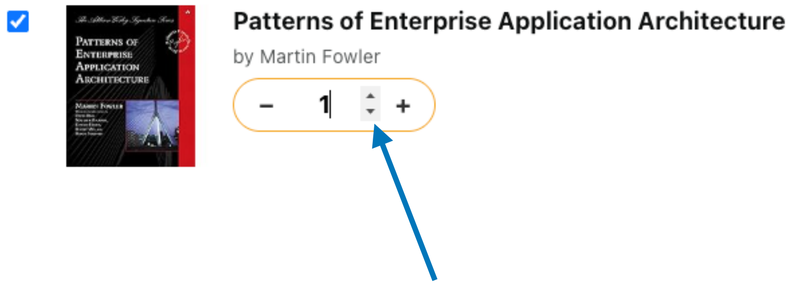


(*) Also called Virtual DOM

Component rendering

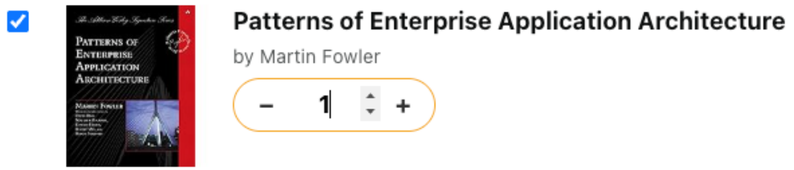
The virtual DOM may not be synchronized with the real DOM

```
export default function QuantitySelectorSpinner(
    { quantity, onChange }: QuantitySelectorProps): ReactNode {

    return (
        <div className={styles.container}>
            <button onClick={() => onChange(quantity - 1)}>-</button>
            <input type="number"
                min={0}
                value={quantity}
            />
            <button onClick={() => onChange(quantity + 1)}>+</button>
        </div>
    );
}
```



Component rendering

```
export default function QuantitySelectorSpinner(
    { quantity, onChange }: QuantitySelectorProps): ReactNode {

    return (
        <div className={styles.container}>
            <button onClick={() => onChange(quantity - 1)}>-</button>
            <input type="number"
                min={0}
                value={quantity}
                onChange={(e) => onChange(e.target.valueAsNumber)}
            />
            <button onClick={() => onChange(quantity + 1)}>+</button>
        </div>
    );
}
```

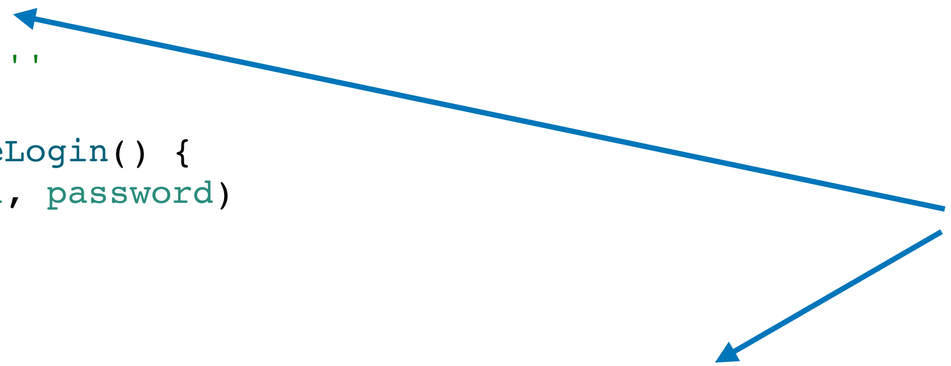


Get the value from the real DOM and update the virtual DOM

Form handling

Form inputs are also subject to desynchronisation between the virtual and real DOM

```
export default function Login() {

    let email = ''
    let password = ''

    function handleLogin() {
        login(email, password)
    }

    return (
        <form>
            <input type="email" id="email" value={email}/>
            <input type="password" id="password" value={password}/>
            <button type="button" onClick={handleLogin}>
                Login
            </button>
        </form>
    )
}
```

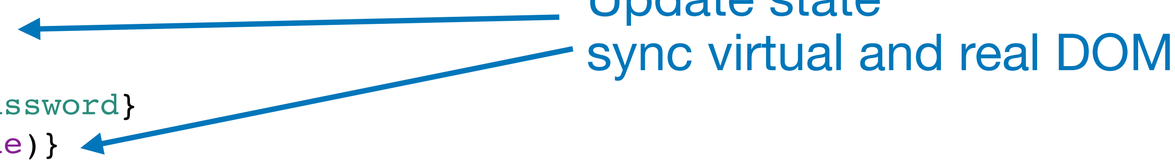


This value won't change when the user types in the email (even though the real input changes)

Form handling - `useState`

Use state to update email and password as the user types

```
export default function Login() {

    const { login } = useContext(SessionContext)!

    const [email, setEmail] = useState('')
    const [password, setPassword] = useState('')

    function handleLogin() {
        login(email, password)
    }

    return (
        <form>
            <input type="email" id="email" value={email}
                onChange={(e) => setEmail(e.target.value)}
            />
            <input type="password" id="password" value={password}
                onChange={(e) => setPassword(e.target.value)}
            />
            <button type="button" onClick={handleLogin}>
                Login
            </button>
        </form>
    )
}
```



Form handling - `useRef`

```
export default function Login() {

    const {login} = useContext(SessionContext)!
    const emailRef = useRef<HTMLInputElement>(null)
    const passwordRef = useRef<HTMLInputElement>(null)

    function handleLogin() {
        login(emailRef.current!.value, passwordRef.current!.value)
    }

    return (
        <form>
            <input ref={emailRef} type="email" id="email"/>
            <input ref={passwordRef} type="password" id="password" />
            <button type="button" onClick={handleLogin}>
                Login
            </button>
        </form>
    )
}
```

useRef is a React hook that allows components to access (real) DOM elements

useRef

`useRef()` stores a mutable value

- persists between renders
- doesn't cause a re-render when it changes

```
const countRef = useRef(0)

function increment() {
    countRef.current++
}
```

doesn't trigger a re-render

Most of the time, it is used to access DOM elements

```
const inputRef = useRef<HTMLInputElement>(null)

function focusInput() { inputRef.current?.focus() }

return <input ref={inputRef} />
```

Virtual DOM <-> Real DOM

Two approaches for “crossing the line” between both DOMs

Controlled Components - `useState`

- Predictable
- Easy validation
- Easy conditional UI
- Easier debugging
- More React-idiomatic

Uncontrolled Components - `useRef`

- Fewer re-renders
- Simpler for basic forms
- Closer to plain HTML behavior
- Harder to synchronize UI logic

Virtual DOM <-> Real DOM

Exercise

	Controlled Componentes	Uncontrolled Components
Source of truth (Virtual DOM / Real DOM)		
Number of re-renders (lower, higher)		
Synchronization between Virtual DOM and Real DOM (easier, harder)		
Debugging (easier, harder)		
Form validation (onSubmit) (easier, harder)		

Virtual DOM <-> Real DOM

	Controlled Componentes	Uncontrolled Components
Source of truth (Virtual DOM / Real DOM)	Virtual DOM	Real DOM
Number of re-renders (lower, higher)	higher	lower
Synchronization between Virtual DOM and Real DOM (easier, harder)	easier	harder
Debugging (easier, harder)	easier	harder
Form validation (onSubmit) (easier, harder)	harder	easier

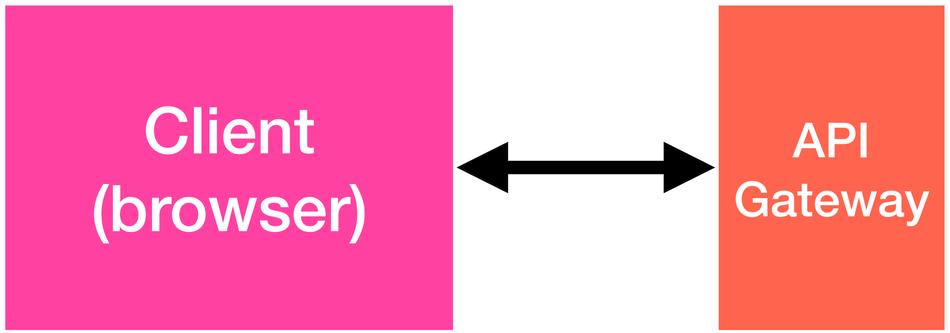
Form error handling

```
export default function Login() {
    const { login } = useContext(SessionContext)!
    const emailRef = useRef<HTMLInputElement>(null)
    const passwordRef = useRef<HTMLInputElement>(null)
    const [error, setError] = useState(false)

    async function handleLogin() {
        const success = await login(emailRef.current!.value, passwordRef.current!.value)

        setError(!success)
    }

    return (
        <form>
            <input ref={emailRef} type="email" id="email" />
            <input ref={passwordRef} type="password" id="password" />

            {error && <p>Invalid credentials</p>}

            <button type="button" onClick={handleLogin}>
                Login
            </button>
        </form>
    )
}
```

To show errors after validation:

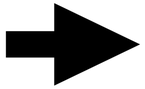
- error message must belong to state
- conditional outputing depending on the existence of a error message

Using REST APIs



fetch

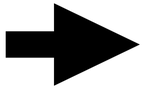
```
GET http://localhost:8080/cars?brand=Toyota&color=red
```



```
fetch('/cars?brand=Toyota&color=red')
```

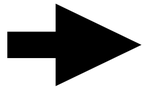
note: by default, the method is GET

```
POST http://localhost:8080/cars
Content-Type: application/json

{
  "plate": "AA-12-BB",
  "brand": "Toyota"
}
```



```
fetch('/cars', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: '{"plate":"AA-12-BB","brand":"Toyota"}'
})
```

```
PATCH http://localhost:8080/cars/42
Authorization: Bearer eyJhbGciOiJIUkpXVCJ9...
Content-Type: application/json

{
  "available": true
}
```



```
fetch('/cars/42', {
    method: 'PATCH',
    headers: {
        'Authorization': 'Bearer eyJhbGciOiJIUkpXVCJ9...',
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        available: true
    })
})
```

fetch

`fetch(…) : Promise<Response>`

fetch is an asynchronous function - you have two options to handle the response

.then() chaining

```
fetch(“/api/message/1”)
        .then(result => result.json())
        .then(data => setMessage(data.message))
        .catch(error => console.error("Failed:", error));
```

await

```
try {
     const result = await fetch(“/api/message/1”);
     const data = await result.json();
     setMessage(data.message);
} catch (error) {
    console.error("Failed:", error);
}
```

fetch

Like in the server side, you should use DTOs to represent the request and response objects

```
interface CreateCarRequestDto {
    readonly plate: string
    readonly brand: string
}

interface CarResponseDto {
    readonly id: number
    readonly plate: string
    readonly brand: string
    readonly createdAt: string
}
```

```
async function createCar(request: CreateCarRequestDto):
                          Promise<CarResponseDto> {
    const response = await fetch('/cars', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(request)
    })

    const data = await response.json()
    return data as CarResponseDto
}
```

marshall the DTO into a (json) string

fetch

Handling response codes

```
async function createCar(
    request: CreateCarRequestDto
): Promise<CarResponseDto> {

    const response = await fetch('/cars', { method: 'POST', … })

    // 201 Created
    if (response.status === 201) {
        const data = await response.json()
        return data as CarResponseDto
    }

    // 400 Bad Request
    if (response.status === 400) {
        const error = await response.json() as ErrorResponseDto
        throw new Error(`Invalid request: ${error.message}`)
    }

    // 401 Unauthorized
    if (response.status === 401) {
        throw new Error('You are not authenticated')
    }
}
```

Using REST APIs

Standard project organization

```
src/
├── components/
│   ├── Button/
│   │   ├── Button.tsx
│   │   └── Button.css
│   └── Input/
│       ├── Input.tsx
│       └── Input.css
│
├── pages/
│   ├── LoginPage.tsx
│   ├── CarsPage.tsx
│   └── CarDetailsPage.tsx
│
├── services/
│   ├── carService.ts
│   ├── carService.dto.ts
│   ├── authService.ts
│   └── authService.dto.ts
│
├── models/
│   └── car.model.ts
│
├── context/
│   └── AuthContext.tsx
│
├── App.tsx
└── main.tsx
```

fetch calls (and their DTOs) are here, outside react components

Using REST APIs

carService.ts

```ts
async function getCarById(id: number): Promise<CarResponseDto> {
    const response = await fetch(`/cars/${id}`)
    const data = await response.json()
    return data as CarResponseDto
}
```

carDetailsPage.tsx

```tsx
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)

    carService.getCarById(42).then(setCar)

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
        </div>
    )
}
```

1. Renders `CarDetailsPage` for the first time
2. Calls the API but since it is asynchronous, continues immediately to the next line
3. Renders “Loading…” since `car` is still null
4. (after receiving the server response) calls `setCar` with the `responseDTO`
5. Re-renders `CarDetailsPage`
6. `Car` is not null, it renders its details

Using REST APIs

carService.ts

```ts
async function getCarById(id: number): Promise<CarResponseDto> {
    const response = await fetch(`/cars/${id}`)
    const data = await response.json()
    return data as CarResponseDto
}
```

carDetailsPage.tsx

```tsx
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)

    carService.getCarById(42).then(setCar)

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
        </div>
    )
}
```

1. Renders `CarDetailsPage` for the first time
2. Calls the API but since it is asynchronous, continues immediately to the next line
3. Renders “Loading…” since `car` is still null
4. (after receiving the server response) calls `setCar` with the `responseDTO`
5. Re-renders `CarDetailsPage`

   5.1. Calls the API again 😢
6. `Car` is not null, it renders its details

infinite loop

6.1. after receiving the server response, re-renders

`useEffect()`

`useEffect()` is a React hook used to run **side effects** after rendering.

A side effect is anything that interacts with something outside React rendering

- HTTP requests
- timers
- subscriptions
- DOM manipulation
- localStorage
- browser APIs

```
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)

    useEffect(() => {
        carService.getCarById(42).then(setCar)
    }, [])

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
        </div>
    )
}
```

useEffect()

```
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)

    useEffect(() => {
        carService.getCarById(42).then(setCar)
    }, [])

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
        </div>
    )
}
```

1. Renders `CarDetailsPage` for the first time
2. Renders “Loading…” since `car` is null
3. Runs the effect - calls the API
4. (after receiving the server response) calls `setCar` with the `responseDTO`
5. Re-renders `CarDetailsPage`
6. `Car` is not null, it renders its details
   Since the effect has already run, it doesn’t call the API again

useEffect()

Controlling **WHEN** Effects Run

the second parameter sets
the dependencies

Run once

```
useEffect(() => {
    console.log('Runs once')
}, [])
```

Run when a value changes

```
useEffect(() => {
    console.log('Count changed')
}, [count])
```

Run after every render

```
useEffect(() => {
    console.log('Runs every render')
})
```

useEffect()

```
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)
    const [needsRefresh, setNeedsRefresh] = useState(true)

    useEffect(() => {
        if (!needsRefresh) return
        carService.getCarById(42)
            .then(car => {
                setCar(car)
                setNeedsRefresh(false)
            })
    }, [needsRefresh])

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
            <button onClick={() => setNeedsRefresh(true)}>Refresh</button>
        </div>
    )
}
```

this block runs when `needsRefresh` changes

clicking the button will trigger a new call to the API

Note: The car id is fixed for didatic purposes

useEffect()

```
export default function CarDetailsPage({carId}: CarDetailsPageProps) {

    const [car, setCar] = useState<CarResponseDto | null>(null)
    const [needsRefresh, setNeedsRefresh] = useState(true)

    useEffect(() => {
        if (!needsRefresh) return
        carService.getCarById(carId)
            .then(car => {
                setCar(car)
                setNeedsRefresh(false)
            })
    }, [carId, needsRefresh])

    if (!car) return <div>Loading...</div>

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
            <button onClick={() => setNeedsRefresh(true)}>Refresh</button>
        </div>
    )
}
```

`carId` is now received via props

`useEffect` now depends on props and state

general rule: the dependency array should include all external values used by the effect (this will be validated by a linter)

useEffect()

```
export default function CarDetailsPage({carId}: CarDetailsPageProps) {
    ...
    useEffect(() => {
        if (!needsRefresh) return
        carService.getCarById(carId)
            .then(car => {
                setCar(car)
                setNeedsRefresh(false)
            })
    }, [needsRefresh])
    ...
}
```

removed the dependency from `carId`

```
function App() {

    const [selectedCarId, setSelectedCarId] = useState(42)

    return (
        <div>
            <button onClick={() => setSelectedCarId(42)}>Load Car 42</button>
            <button onClick={() => setSelectedCarId(99)}>Load Car 99</button>
            <CarDetailsPage carId={selectedCarId} />
        </div>
    )
}
```

This won't fetch data from car 99

```tsx
export default function CounterCard({initial}: { initial: number }) {

    const [count, setCount] = useState(initial)
    const [double, setDouble] = useState(initial * 2)

    console.log('render', {initial, count, double})

    useEffect(() => {
        console.log('effect A', {initial, count})
    }, [count, double, initial])

    useEffect(() => {
        console.log('effect B', {count, double})
    }, [double])

    return (
        <div>

            <h1>{count} / {double}</h1>

            <button onClick={() => {
                setCount(count + 1);
                console.log('after setCount', {count, double})
            }}>Increment Count</button>

            <button onClick={() => {
                setDouble(double + 2);
                console.log('after setDouble', {count, double})
            }}>Increment Double</button>

        </div>
    )
}
```

Exercise

```tsx
<CounterCard initial={3} />
```

User clicks “Increment Count” and then “Increment Double”

What is printed in the console (since it is first rendered)?

```tsx
export default function CounterCard({initial}: { initial: number }) {

    const [count, setCount] = useState(initial)
    const [double, setDouble] = useState(initial * 2)

    console.log('render', {initial, count, double})

    useEffect(() => {
        console.log('effect A', {initial, count})
    }, [count, double, initial])

    useEffect(() => {
        console.log('effect B', {count, double})
    }, [double])

    return (
        <div>

            <h1>{count} / {double}</h1>

            <button onClick={() => {
                setCount(count + 1);
                console.log('after setCount', {count, double})
            }}>Increment Count</button>

            <button onClick={() => {
                setDouble(double + 2);
                console.log('after setDouble', {count, double})
            }}>Increment Double</button>

        </div>
    )
}
```

Resolution

```tsx
<CounterCard initial={3} />
```

User clicks “Increment Count” and then “Increment Double”

What is printed in the console (since it is first rendered)?

```
render { 3, 3, 6 }
effect A { 3, 3 }
effect B { 3, 6 }
(clicks “Increment Count”)
after setCount { 3, 6 }
render { 3, 4, 6 }
effect A { 3, 4 }
(clicks “Increment Count”)
after setDouble { 4, 6 }
render { 3, 4, 8 }
effect A { 3, 4 }
effect B { 4, 8 }
```

Error handling

carService.ts

```
async function getCarById(id: number): Promise<CarResponseDto> {
    const response = await fetch(`/cars/${id}`)
    const data = await response.json()
    return data as CarResponseDto
}
```

Possible errors:

- No connection - fetch throws Exception
- Server responds with code <> 200 - you have to manually check response code

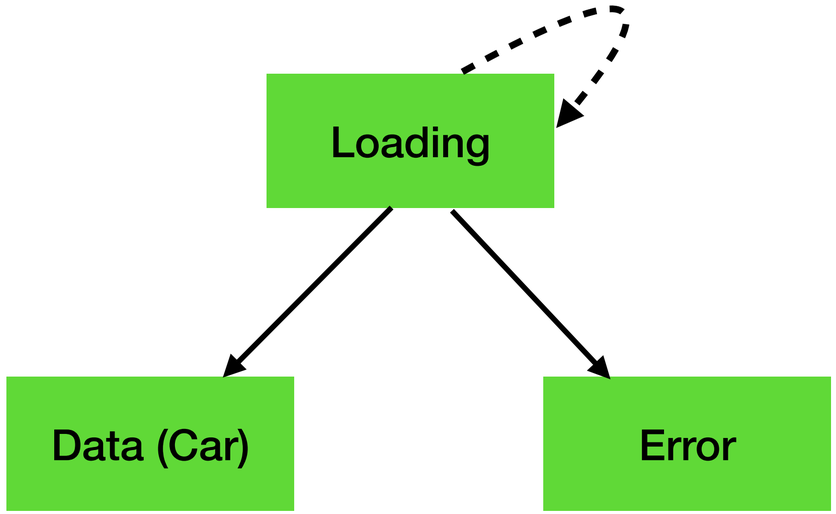
Error handling

carService.ts

```
async function getCarById(id: number): Promise<CarResponseDto> {
    const response = await fetch(`/cars/${id}`)

    if (!response.ok) {
        throw new Error(`Failed to fetch car (${response.status})`)
    }

    const data = await response.json()

    return data as CarResponseDto
}
```

Async handling

```
export default function CarDetailsPage() {

    const [car, setCar] = useState<CarResponseDto | null>(null)
    const [loading, setLoading] = useState(true)
    const [error, setError] = useState<string | null>(null)

    useEffect(() => {
        setLoading(true)
        setError(null)

        carService
            .getCarById(42)
            .then(car => { setCar(car) })
            .catch(error => { setError(error.message) })
            .finally(() => { setLoading(false) })
    }, [])

    if (loading) { return <div>Loading...</div> }
    if (error) { return <div>Error: {error}</div> }

    return (
        <div>
            <h1>Car Details</h1>
            <p>{car.brand}</p>
            <p>{car.plate}</p>
        </div>
    )
}
```



We have to manage 3 state variables

useReducer

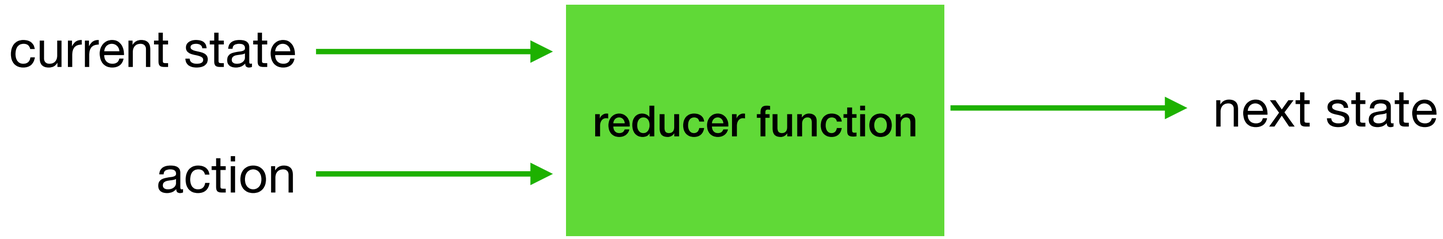
`useReducer` is a Hook for managing complex state logic (alternative to `useState`)

1. Define a reducer function
2. Dispatch actions
3. React updates state through the reducer (re-rendering the component)

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- `state`: current state
- `dispatch`: function to send actions
- `reducer`: function describing state changes
- `initialState`: starting value

reducer function




```
type Action =
    | { type: "increment" }
    | { type: "decrement" }

function reducer(state: number, action: Action): number {

    switch (action.type) {

       case "increment":
            return state + 1

        case "decrement":
            return state - 1

        default:
            return state
    }
}}
```

TypeScript **union type** - Action can be
{ type: "increment" } or { type: "increment" }

```
reducer(0, { type: "increment" }) // 1
reducer(1, { type: "increment" }) // 2
reducer(2, { type: "decrement" }) // 1
```

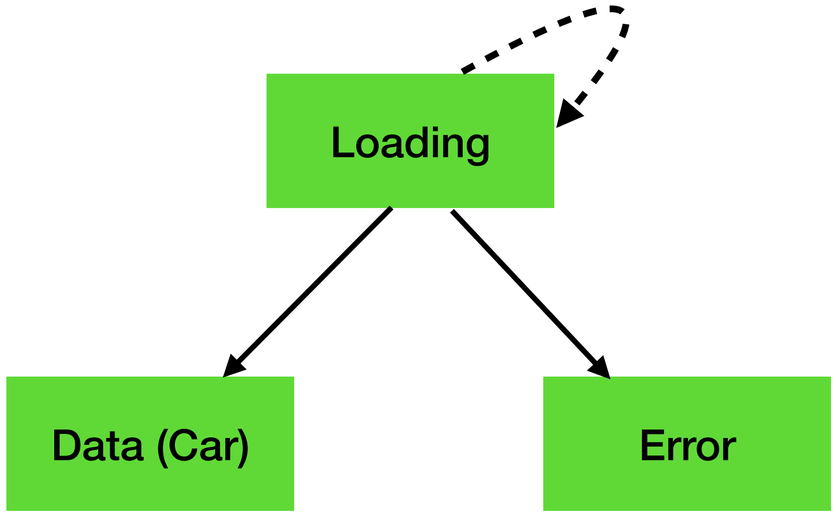
reducer function

```
export default function Counter() {

    const [count, dispatch] = useReducer(reducer, 0)

    return (
        <div>
            <h1>{count}</h1>

            <button onClick={() => dispatch({ type: "increment" }) } >
                Increment
            </button>

            <button
                onClick={() => dispatch({ type: "decrement" }) } >
                Decrement
            </button>
        </div>
    )
}
```

1. Pass an action to the reducer
2. The reducer calculates the new state
3. Re-renders the component

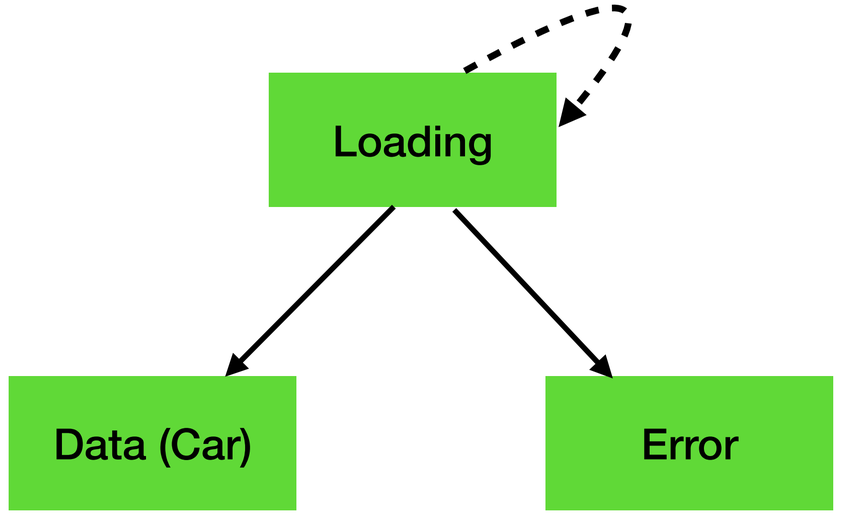
Async handling with reducer

```ts
type State =
    | { status: "loading" }
    | { status: "success"; car: CarResponseDto }
    | { status: "error"; error: string }

type Action =
    | { type: "FETCH_SUCCESS"; payload: CarResponseDto }
    | { type: "FETCH_ERROR"; payload: string }

function reducer(state: State, action: Action): State {

    switch (action.type) {

        case "FETCH_SUCCESS":
            return { status: "success", car: action.payload }

        case "FETCH_ERROR":
            return { status: "error", error: action.payload }

        default:
            return state
    }
}
```



Async handling with reducer

```
export default function CarDetailsPage() {

    const [state, dispatch] = useReducer(reducer, { status: "loading" })

    useEffect(() => {
        carService
            .getCarById(42)
            .then(car => {
                dispatch({ type: "FETCH_SUCCESS", payload: car })
            })
            .catch(error => {
                dispatch({ type: "FETCH_ERROR", payload: error.message })
            })
    }, [])

    if (state.status === "loading") { return <div>Loading...</div> }
    if (state.status === "error") { return <div>Error: {state.error}</div> }

    return (
        <div>
            <h1>Car Details</h1>
            <p>{state.car.brand}</p>
            <p>{state.car.plate}</p>
        </div>
    )
}
```

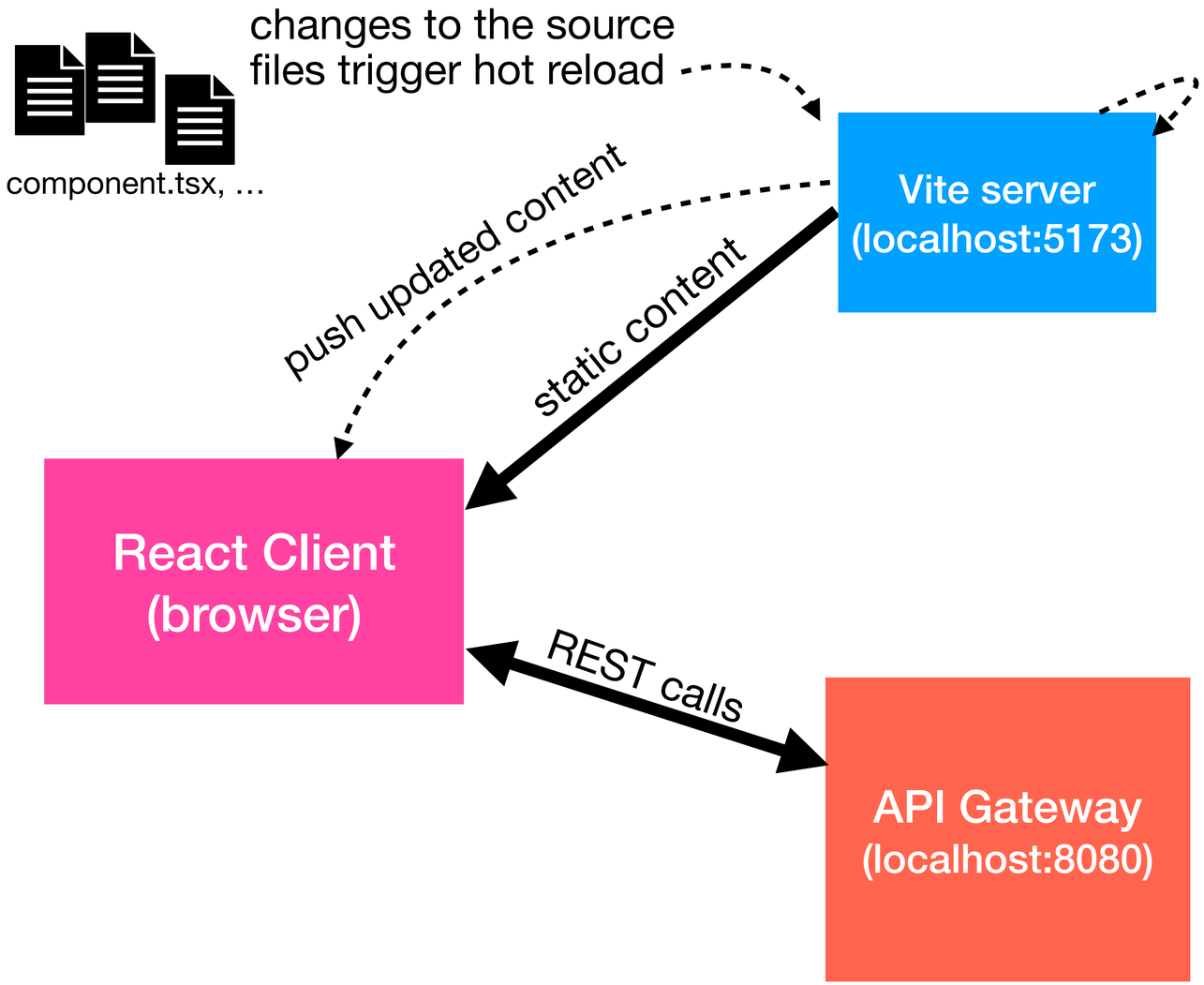


We have to manage only 1 state variable

useState vs useReducer

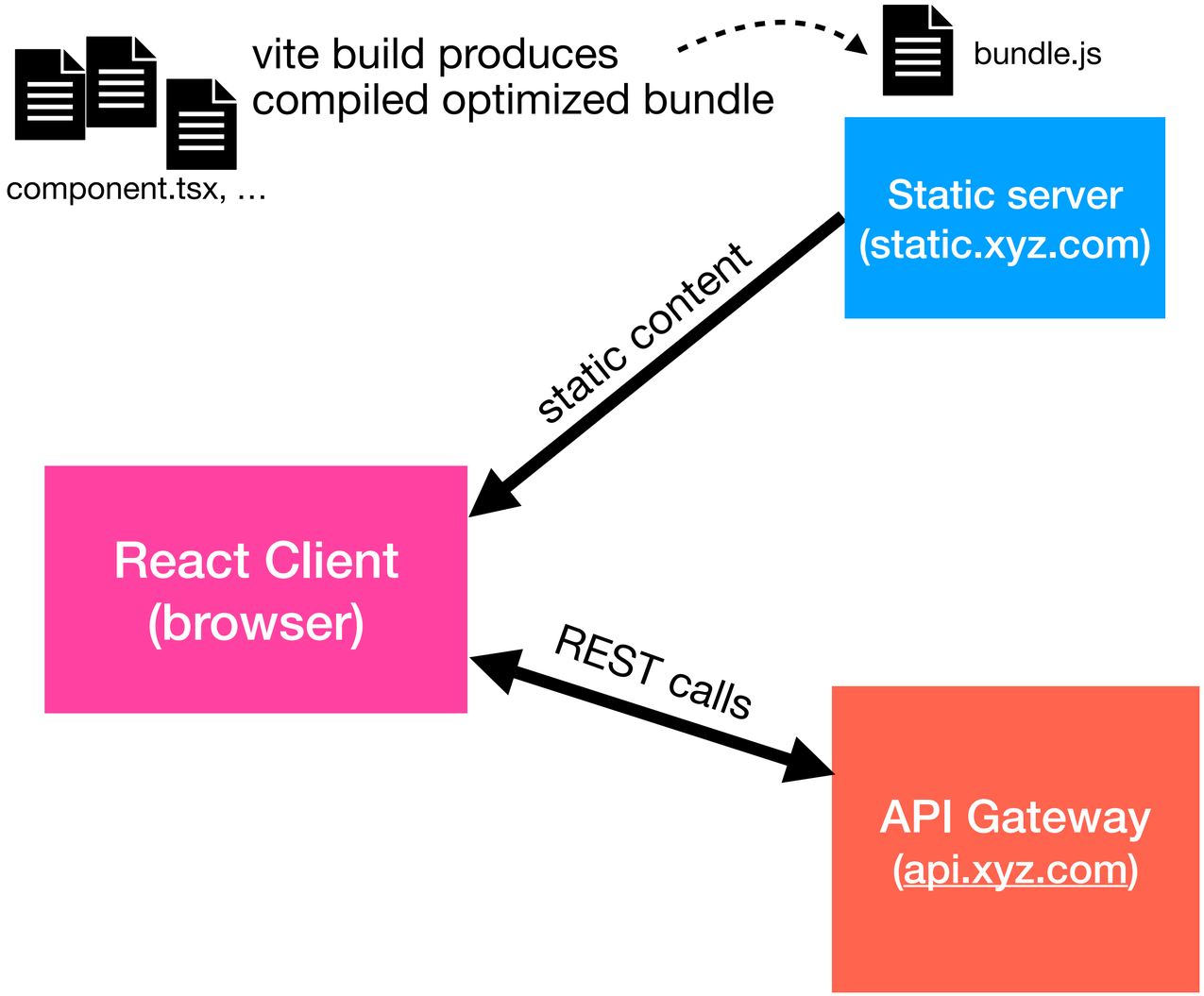
	useState	useReducer
state	primitive type	object
update state	set value	dispatch action
state transition logic	scattered	centralized
best for	simple state (e.g., counter)	complex state logic (e.g., async states)

Architecture (dev)



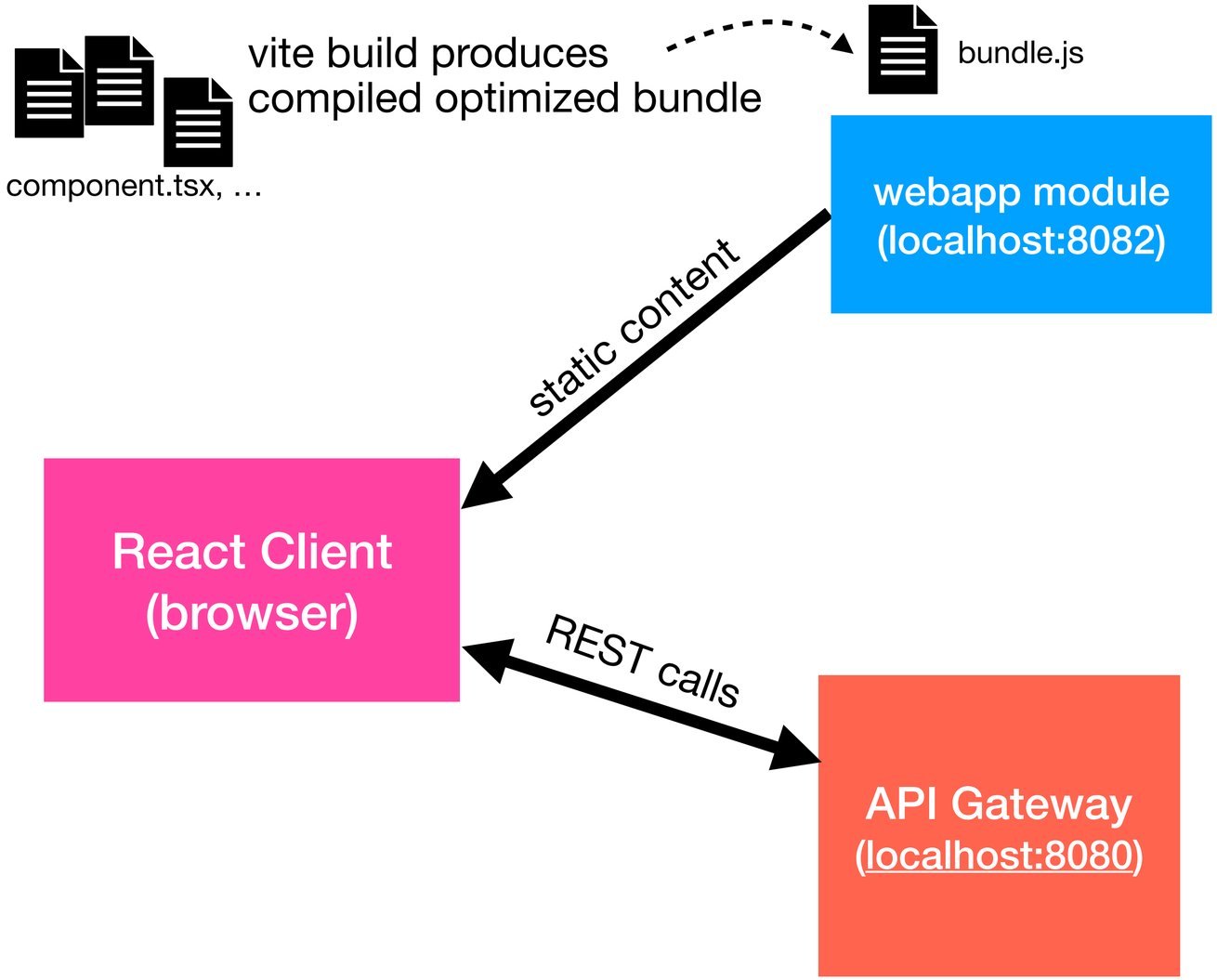
1. user navigates to localhost:5173
2. vite server returns index.html and transpiled js code
3. browser runs js code (rendering the app)
4. js code calls API gateway…

Architecture (prod)



1. developer builds a bundle and installs it in the static server
2. user navigates to static.xyz.com
3. static server returns index.html and bundle
4. browser runs js code (rendering the app)
5. js code calls API gateway…

Architecture (project)



The webapp module will serve:

- the “react client” static files
- SSR pages

Differentiate by path to prevent the double routing problem: `/app` and `/web`

Architecture (project)

SSR pages (webapp) must live inside `/web`

```
@RequestMapping("/web/flights")
```

```
return "redirect:/web/flights"
```

```
@Bean
fun securityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        ...
        .authorizeHttpRequests {
            it.requestMatchers("/web/flights/new")...
            ...
```

Architecture (project)

React Client must live inside `/app`

```
@Bean
fun securityFilterChain(http: HttpSecurity):
      SecurityFilterChain {
    http
       .authorizeHttpRequests {
            it.requestMatchers("/app/**").permitAll()
```

`vite.config.ts`

```
export default defineConfig({
  plugins: [react()],
  base: process.env.VITE_BASE ?? '/',
})
```

sets the public base path for the app when served in production - based on the env variable VITE_BASE

Also, configure SSR module to respond with index.html for all /app/* requests (e.g., /app/login) - explained in the slides about the double route problem

Environment specific properties

Create environment specific files in the project root folder:

.env.development

```
VITE_API_GATEWAY_URL=http://localhost:8080
```

.env.production

```
VITE_API_GATEWAY_URL=https://api.xyz.com
```

```
const API_GATEWAY_URL = import.meta.env.VITE_API_GATEWAY_URL

fetch(`${API_GATEWAY_URL}/cars`)
```

This is replaced by the correct property by Vite

Build production bundle

```
pnpm build
```

- **Module bundling** - Combine many files into optimized bundles
- **Transpilation** - Convert alternative syntaxes (Typescript, JSX, …) into compatible Javascript
- **Optimization for production** - minification, code splitting, compression, …

Output goes to the `dist` folder -> should be copied to the static server

Integration with maven

The maven sub-module responsible for static server (e.g., SSR web app) should automate the react build and resource copying

```
<profiles>
    <profile>
        <id>frontend</id>
        <build>
            <plugins>
                <plugin>
                    <groupId>com.github.eirslett</groupId>
                    <artifactId>frontend-maven-plugin</artifactId>
                    ...
                </plugin>
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-resources-plugin</artifactId>
                    ...
                </plugin>
            </plugins>
        </build>
    </profile>
```

execute only on profile “frontend”

1. Downloads its own Node and `pnpm` into the project directory
2. Allows calling `pnpm` commands

Copies the pnpm build output to `/src/main/static` (automatically served by spring)

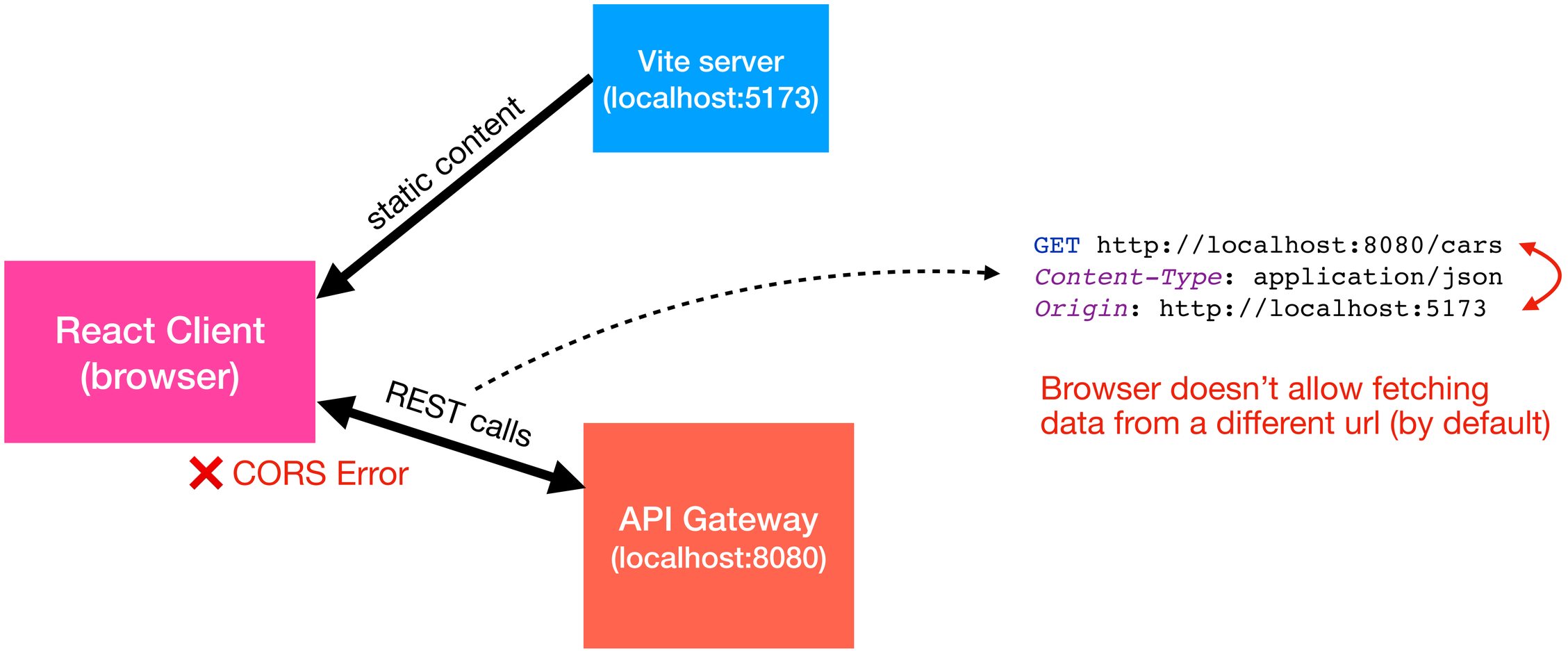
Integration with maven

```xml
<plugin>
    <groupId>com.github.eirslett</groupId>
    <artifactId>frontend-maven-plugin</artifactId>
    <version>1.15.1</version>
    <configuration>
        <workingDirectory>${project.basedir}/../react-client</workingDirectory>
        <nodeVersion>v22.12.0</nodeVersion>
        <pnpmVersion>v9.15.0</pnpmVersion>
    </configuration>
    <executions>
        <execution>
            <id>install-node-and-pnpm</id>
            <goals>
                <goal>install-node-and-pnpm</goal>
            </goals>
            <phase>generate-resources</phase>
        </execution>
        <execution>
            <id>pnpm-install</id>
            <goals>
                <goal>pnpm</goal>
            </goals>
            <phase>generate-resources</phase>
            <configuration>
                <arguments>install</arguments>
            </configuration>
        </execution>
        <execution>
            <id>pnpm-build</id>
            <goals>
                <goal>pnpm</goal>
            </goals>
            <phase>generate-resources</phase>
            <configuration>
                <arguments>build</arguments>
                <environmentVariables>
                   <VITE_BASE>/app/</VITE_BASE>
                </environmentVariables>
            </configuration>
        </execution>
    </executions>
</plugin>
```

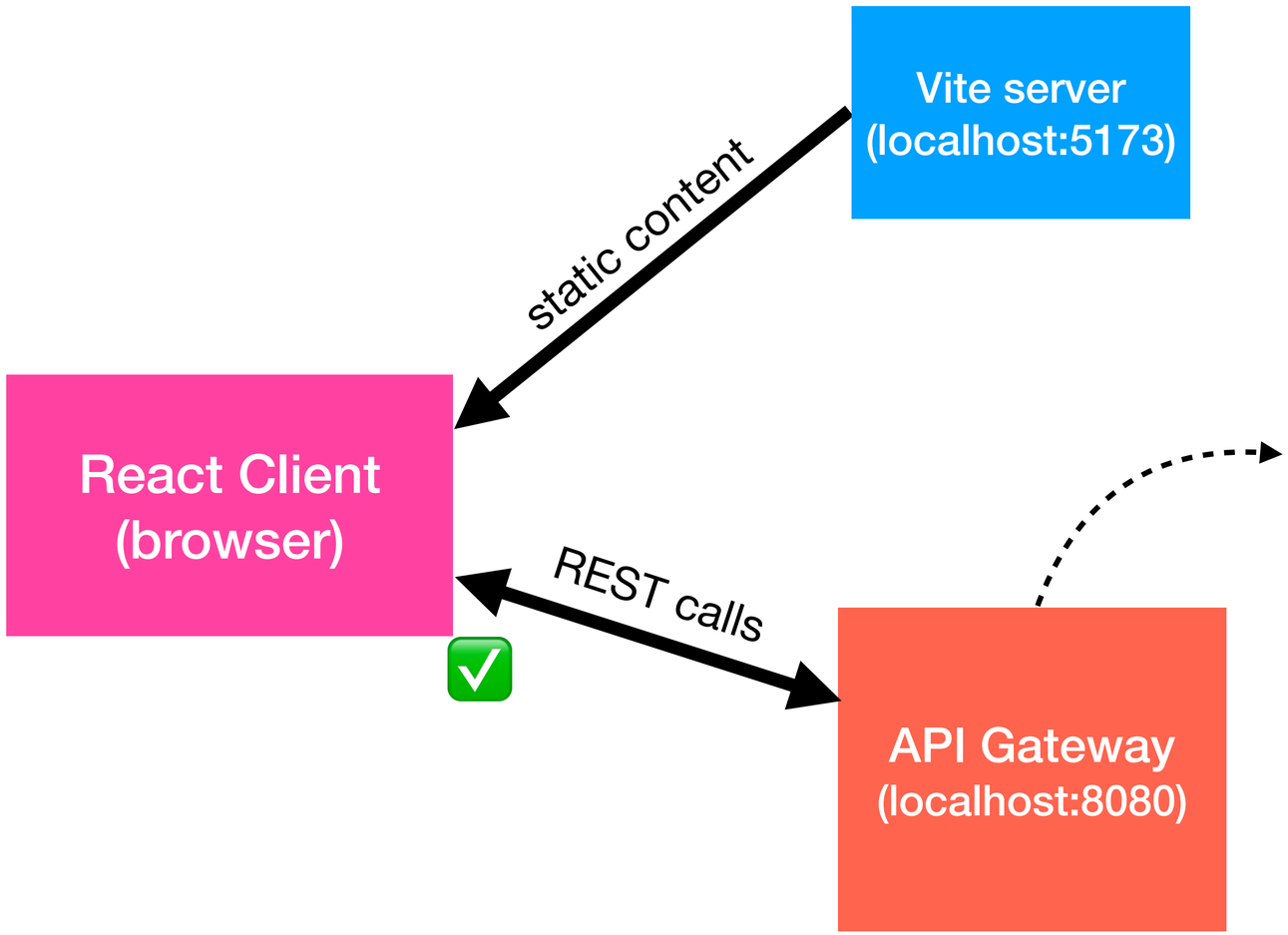
Integration with maven

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-resources-plugin</artifactId>
    <executions>
        <execution>
            <id>copy-react-bundle</id>
            <phase>compile</phase>
            <goals>
                <goal>copy-resources</goal>
            </goals>
            <configuration>
                <outputDirectory>${project.basedir}/src/main/resources/static/app</outputDirectory>
                <resources>
                    <resource>
                        <directory>${project.basedir}/../react-client/dist</directory>
                    </resource>
                </resources>
            </configuration>
        </execution>
    </executions>
</plugin>
```

CORS



CORS



Register a filter that sets allowed origins

```
@Configuration
class WebConfig() : WebMvcConfigurer {

    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    fun corsFilter(): CorsFilter {
        val config = CorsConfiguration()
        config.allowedOrigins = listOf("http://localhost:5173",
                                       "https://static.xyz.com")
        config.allowedMethods = listOf("GET", "POST", "PUT",
                                       “DELETE", "OPTIONS")
        config.allowedHeaders = listOf("*")
        config.allowCredentials = true
        val source = UrlBasedCorsConfigurationSource()
        source.registerCorsConfiguration("/**", config)
        return CorsFilter(source)
    }
}
```

CORS

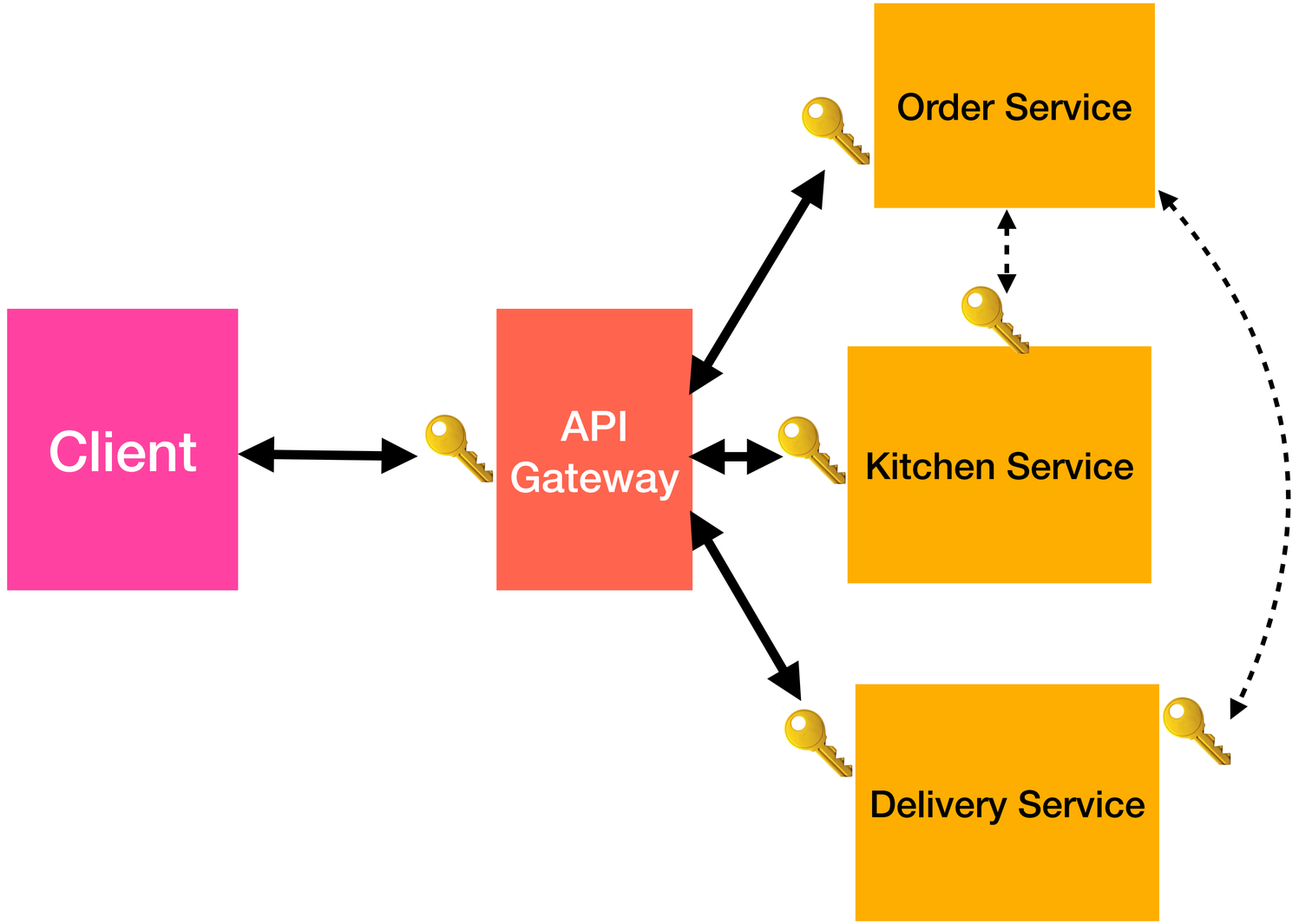
Testing CORS through HTTP requests (Intellij)

```
### CORS preflight (simulates browser OPTIONS before GET from localhost:5173)
# Expect: 200 with Access-Control-Allow-Origin: http://localhost:5173
OPTIONS http://localhost:8080/cars
Origin: http://localhost:5173
Access-Control-Request-Method: GET
Access-Control-Request-Headers: Content-Type


### CORS request (simulates browser GET from localhost:5173)
# Expect: 200 with Access-Control-Allow-Origin: http://localhost:5173
GET http://localhost:8080/cars
Content-Type: application/json
Origin: http://localhost:5173
```

Microservice architecture

Authentication

Revisions from week 8



Users must be authenticated to enter the front gate (API Gateway)

- How do they authenticate?
- How is their identity propagated across the system?

Microservice architecture

Authentication

Revisions from week 8

- The client application connects to an IdP (Identity Provider) which can be internal or external
- The IdP validates credentials and returns a token
- Calls API Gateway (or backend) with the token
- Gateway / service validates token
- Request forwarded to microservices (including token)

Note: If the IdP is external, the external token may be converted to an internal token

CSR applications

Authentication

1. App renders a login (form) page
2. Upon clicking the submit button, the app POSTs the credentials to the API Gateway (using fetch())
3. If success, the API Gateway returns a JSON payload with a JWT token
4. App stores the token
5. App sets state (UserSession) to trigger a re-render (probably triggering a new route)
6. All further requests include the token in the Authorization header

CSR applications

Storing tokens

Tokens can be stored:

- In memory
  - Lost on page refresh
  - Lost when browser tab closes
  - Most secure

```
const [token, setToken] = useState(null)
```

- Session Storage
  - Scoped to one browser tab
  - Lost when browser tab closes
  - Survives page refresh
  - Vulnerable to XSS attacks

```
sessionStorage.setItem("token", token);
```

- Local Storage
  - Persistent
  - Vulnerable to XSS attacks

```
localStorage.setItem("token", token);
```

- Cookies
  - Persistent
  - Protected from JS Access (if HttpOnly)

CSR applications

Storing tokens

Setting initial state with the value stored in sessionStorage (if it exists)

```
function loadSession(): UserSession | undefined {
    const stored = sessionStorage.getItem(SESSION_KEY)
    return stored ? JSON.parse(stored) : undefined
}

const [user, setUser] = useState<UserSession | undefined>(loadSession)
```

Updating sessionStorage before setting the new state

```
function setUserSession(user: UserSession | undefined) {
    if (user) sessionStorage.setItem(SESSION_KEY, JSON.stringify(user))
    else sessionStorage.removeItem(SESSION_KEY)
    setUser(user)
}
```

CSR applications

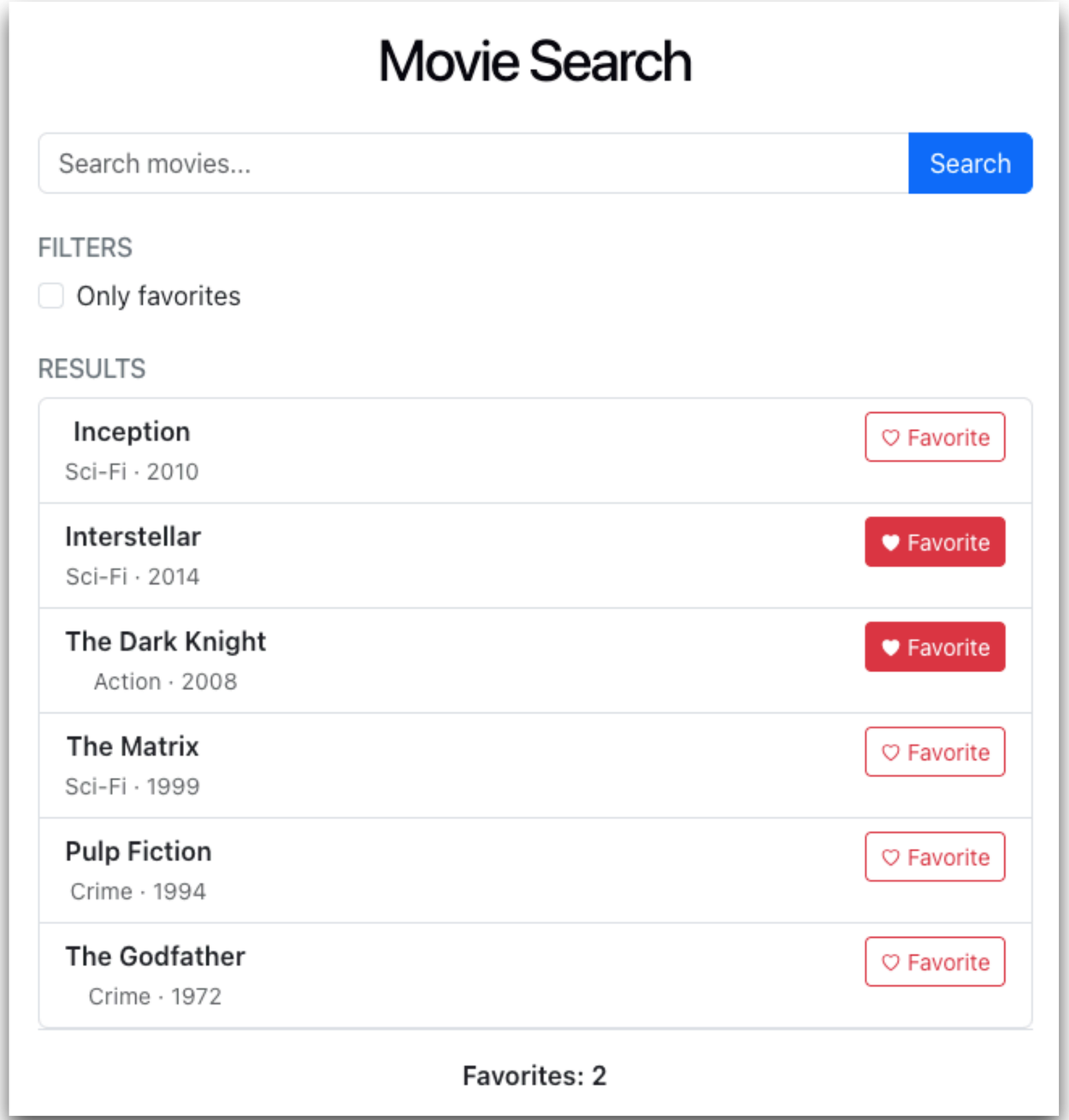
API Key

API Keys allow the server to identify the client application and prevent API abuse

This is great for mobile/native applications, not so great for browser application (API key is exposed)

Two options:

- Backend-for-Frontend (BFF) proxy - a thin server between the browser and the API gateway (the BFF proxy attaches the API key)
- Rely solely on user tokens (only works for authenticated pages)



Exercise

Draw a **component tree diagram** representing how you would decompose the page into reusable React components, in the most efficient way possible, using controlled components

For each component, indicate:

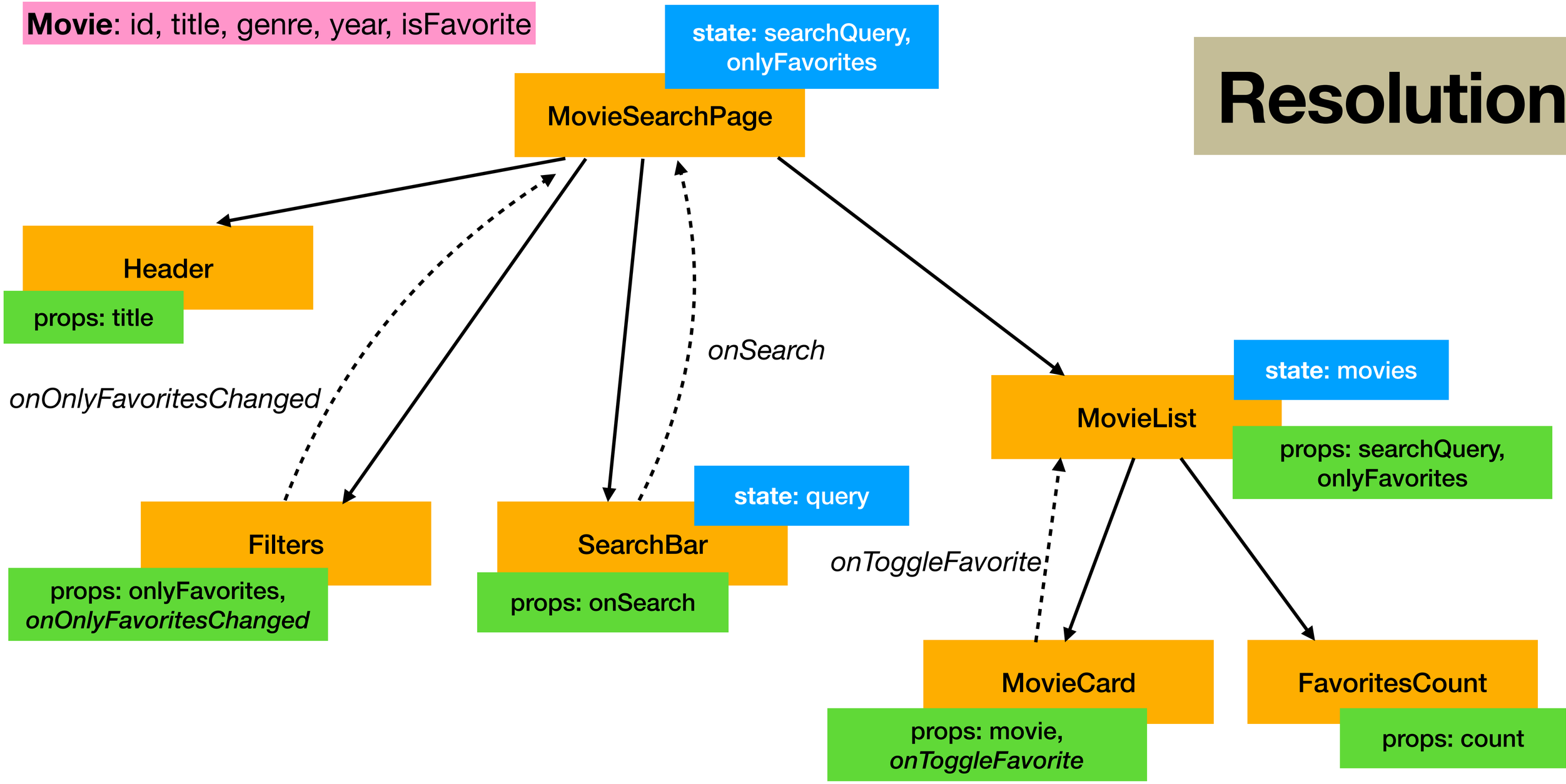
- Component name
- State variables owned by the component (if any)
- Props received from the parent component (if any)
- Callbacks called on parent component (if any), next to an ↑ arrow

Also indicate the object structures if needed

Do not write code

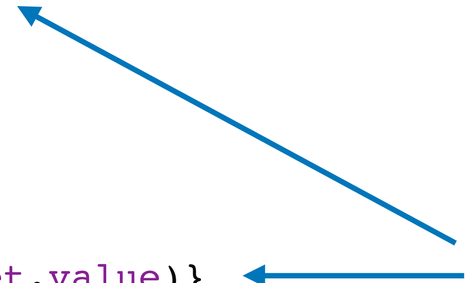
Resolution

Movie: id, title, genre, year, isFavorite



Resolution

```tsx
interface SearchBarProps {
  onSearch: (query: string) => void
}

export default function SearchBar({ onSearch }: SearchBarProps) {
  const [query, setQuery] = useState('');

  return (
    <div>
      <input type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <button onClick={() => onSearch(query)}>
        Search
      </button>
    </div>
  );
}
```



Controlled component: always in sync with real DOM

Note: code was not needed for this exercise, it is shown just to help understand the behaviour

Resolution

```
interface MovieListProps {
  searchQuery: string;
  onlyFavorites: boolean;
}

export default function MovieList({ searchQuery, onlyFavorites }: MovieListProps) {
  const [movies, setMovies] = useState<Movie[]>(ALL_MOVIES);

  function handleToggleFavorite(id: number) {
    setMovies(movies.map((m) => m.id === id ? { ...m, isFavorite: !m.isFavorite } : m));
  }

  const query = searchQuery.trim().toLowerCase();
  const displayed = movies.filter((m) =>
      m.title.toLowerCase().includes(query) &&
      (!onlyFavorites || m.isFavorite));

  return (
    <div>
      <h2>Results</h2>
      <ul className="list-group">
        {displayed.map((movie) => (
          <MovieCard
            key={movie.id}
            movie={movie}
            onToggleFavorite={handleToggleFavorite}
          />
        ))}
      </ul>
      <FavoritesCount count={movies.filter((m) => m.isFavorite).length} />
    </div>
  );
}
```